LIBRARY MANAGEMENT SYSTEM

DSA Mini Project Using C Language

Project type: Console-based application

Core concepts: Structures, arrays, linear search, functions, and menu-driven programming

Purpose: A beginner-friendly project suitable for learning and uploading to GitHub.

Prepared as a starter project — customize and test it before submitting or publishing.

1. Project Overview

The Library Management System is a simple C program that helps manage a small collection of books. A user can add books, display all books, search by book ID, issue a book, and return a book.

2. Data Structures and Concepts Used

Structure (struct): Stores each book's ID, title, author, and availability status together.

Array: The array `books` stores up to 100 book records.

Linear search: The `findBook()` function checks each record until the requested ID is found.

Functions: Separate functions keep each operation organized and easier to maintain.

3. Features

- Add a new book with a unique ID
- Display all books and their status
- Search for a book by ID
- Issue an available book
- Return an issued book
- Validate duplicate IDs and common invalid choices

4. Source Code

Save the following code as `library_management.c`.

```
#include <stdio.h>
#include <string.h>

#define MAX_BOOKS 100

typedef struct {
    int id;
    char title[100];
    char author[100];
    int available; /* 1 = available, 0 = issued */
} Book;

Book books[MAX_BOOKS];
int bookCount = 0;

/* Linear search: O(n) */
int findBook(int id) {
    for (int i = 0; i < bookCount; i++) {
        if (books[i].id == id) return i;
    }
    return -1;
}

void addBook(void) {
    if (bookCount >= MAX_BOOKS) {
        printf("Library is full.\\n");
        return;
    }

    Book b;
    printf("Enter book ID: ");
    if (scanf("%d", &b.id) != 1) return;

    if (findBook(b.id) != -1) {
        printf("A book with this ID already exists.\\n");
        return;
    }

    printf("Enter title (one word): ");
    scanf("%99s", b.title);
    printf("Enter author (one word): ");
    scanf("%99s", b.author);

    b.available = 1;
    books[bookCount++] = b;
    printf("Book added successfully.\\n");
}
```

```
void displayBooks(void) {
    if (bookCount == 0) {
        printf("No books in the library.\\n");
        return;
    }

    printf("\\nID\\tTitle\\tAuthor\\tStatus\\n");
    for (int i = 0; i < bookCount; i++) {
        printf("%d\\t%s\\t%s\\t%s\\n",
               books[i].id, books[i].title, books[i].author,
               books[i].available ? "Available" : "Issued");
    }
}

void searchBook(void) {
    int id;
    printf("Enter book ID to search: ");
    if (scanf("%d", &id) != 1) return;

    int index = findBook(id);
    if (index == -1) {
        printf("Book not found.\\n");
    } else {
        printf("Found: ID=%d, Title=%s, Author=%s, Status=%s\\n",
               books[index].id, books[index].title, books[index].author,
               books[index].available ? "Available" : "Issued");
    }
}

void issueBook(void) {
    int id;
    printf("Enter book ID to issue: ");
    if (scanf("%d", &id) != 1) return;

    int index = findBook(id);
    if (index == -1) {
        printf("Book not found.\\n");
    } else if (!books[index].available) {
        printf("This book is already issued.\\n");
    } else {
        books[index].available = 0;
        printf("Book issued successfully.\\n");
    }
}

void returnBook(void) {
    int id;
    printf("Enter book ID to return: ");
    if (scanf("%d", &id) != 1) return;

    int index = findBook(id);
    if (index == -1) {
        printf("Book not found.\\n");
    } else if (books[index].available) {
        printf("This book was not issued.\\n");
    } else {
        books[index].available = 1;
        printf("Book returned successfully.\\n");
    }
}

int main(void) {
    int choice;

    do {
        printf("\\n===== LIBRARY MANAGEMENT SYSTEM =====\\n");
        printf("1. Add book\\n");
        printf("2. Display all books\\n");
        printf("3. Search book by ID\\n");
        printf("4. Issue book\\n");
        printf("5. Return book\\n");
        printf("0. Exit\\n");
        printf("Enter choice: ");

        if (scanf("%d", &choice) != 1) {
            printf("Invalid input. Exiting.\\n");
            break;
        }

        switch (choice) {
            case 1: addBook(); break;
            case 2: displayBooks(); break;
            case 3: searchBook(); break;
            case 4: issueBook(); break;
            case 5: returnBook(); break;
            case 0: printf("Thank you for using the system.\\n"); break;
            default: printf("Invalid choice. Try again.\\n");
        }
    } while (choice != 0);

    return 0;
}
```

5. Compile and Run

Using GCC on Linux, macOS, or Windows with GCC installed:

```
gcc library_management.c -o library_management
./library_management
```

On Windows Command Prompt, run `library_management.exe` after compiling.

6. Sample Run

```
===== LIBRARY MANAGEMENT SYSTEM =====
1. Add book
2. Display all books
3. Search book by ID
4. Issue book
5. Return book
0. Exit
Enter choice: 1
Enter book ID: 101
Enter title (one word): DSA
Enter author (one word): Sharma
Book added successfully.

Enter choice: 2
ID      Title   Author  Status
101     DSA     Sharma  Available

Enter choice: 0
Thank you for using the system.
```

7. Time Complexity

Add book: O(n) in this simple version because it checks for a duplicate ID using linear search.
Search book: O(n).
Issue/return book: O(n), because the program first searches for the book.
Display books: O(n).

Here, n is the number of books stored.

8. Limitations and Possible Improvements

• Book titles and author names currently accept one word only.
• Data is stored in memory and is lost when the program exits.
• This starter project does not track borrowers, due dates, or fines.

Possible upgrades: use file handling to save records, accept full names with spaces, add member records, track issue dates, and implement sorting or a linked list.

9. GitHub Publishing Checklist

1. Create a repository named `library-management-system-c`.
2. Upload `library_management.c` and this PDF.
3. Add a README with the project description, features, compile instructions, and sample output.
4. Test the code on your compiler and update the README if you make changes.

Suggested repository description: “A beginner-friendly Library Management System in C demonstrating structures, arrays, linear search, and menu-driven programming.”

Note: This project is a simple educational starter, not a production-ready library database.